

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

단원 3 : 딥러닝

DAY8. 딥러닝 오버피팅 방지 기법

목 차

1. 딥러닝 오버피팅 방지 기법		3
2. 딥러닝 최적 모델 학습		14

1. 딥러닝 오버피팅 방지 기법

딥러닝에서 **오버피팅(Overfitting, 과적합)** 은 모델이 학습 데이터는 매우 잘 맞추지만, 새로운 데이터에는 성능이 떨어지는 현상입니다.

즉, 모델이 **일반적인 패턴**을 학습한 것이 아니라, 학습 데이터의 **세부 특징, 노이즈, 우연한 패턴**까지 외워버린 상태입니다.

1. 오버피팅이 발생하는 이유

딥러닝 모델은 많은 파라미터를 가지고 있기 때문에 표현력이 매우 큽니다. 표현력이 크다는 것은 복잡한 문제를 잘 학습할 수 있다는 장점이 있지만, 반대로 학습 데이터가 부족하거나 모델이 너무 복잡하면 데이터를 그대로 외워버릴 수 있습니다.

예를 들어 고양이와 강아지를 분류하는 모델을 학습한다고 가정하면, 모델은 원래 귀 모양, 얼굴 형태, 털 패턴 같은 일반적인 특징을 학습해야 합니다. 그런데 학습 데이터에 있는 특정 사진의 배경, 조명, 촬영 각도까지 외워버리면 새로운 사진에서는 성능이 떨어집니다.

2. 오버피팅 확인 방법

오버피팅은 보통 **학습 손실과 검증 손실의 차이**로 확인합니다.

구분	학습 데이터 성능	검증 데이터 성능	의미
정상 학습	좋음	좋음	일반화가 잘 됨
오버피팅	매우 좋음	나쁨	학습 데이터를 외움
언더피팅	나쁨	나쁨	모델이 충분히 학습하지 못함

대표적인 형태는 다음과 같습니다.

Epoch 증가
Train Loss ↓ 계속 감소
Validation Loss ↓ 감소하다가 다시 증가

이 경우 모델은 학습 데이터에 점점 더 맞춰지고 있지만, 검증 데이터에 대한 일반화 성능은 나빠지고 있는 것입니다.

3. 오버피팅 방지 기법

3.1 데이터 증강(Data Augmentation)

데이터 증강은 기존 데이터를 변형해서 새로운 학습 데이터처럼 사용하는 방법입니다.

이미지 데이터에서는 다음과 같은 방법을 사용합니다.

방법	설명
회전	이미지를 약간 회전
좌우 반전	이미지를 가로로 뒤집음
확대/축소	이미지 크기 변형
밝기 조절	조명 변화 반영
자르기	이미지 일부 영역 사용
노이즈 추가	약간의 잡음 추가

예를 들어 고양이 사진 1장이 있을 때, 이를 회전하거나 밝기를 바꾸면 모델 입장에서는 다양한 고양이 사진을 학습하는 효과가 생깁니다.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# 이미지 데이터를 학습 중에 다양한 형태로 변형하기 위한 객체 생성
datagen = ImageDataGenerator(
    rotation_range=20,      # 이미지를 최대 20도까지 회전
    width_shift_range=0.1,  # 이미지를 좌우 방향으로 10% 이동
    height_shift_range=0.1, # 이미지를 상하 방향으로 10% 이동
    zoom_range=0.1,        # 이미지를 최대 10% 확대 또는 축소
    horizontal_flip=True    # 이미지를 좌우 반전
)
```

데이터 증강은 모델이 특정 이미지 형태만 외우지 않고 다양한 상황에서도 잘 작동하도록 도와줍니다.

3.2 드롭아웃(Dropout)

드롭아웃은 학습 중에 일부 뉴런을 무작위로 꺼버리는 방법입니다.

예를 들어 신경망에 100개의 뉴런이 있다면, 학습할 때마다 일부 뉴런을 임시로 사용하지 않습니다. 이렇게 하면 특정 뉴런에 지나치게 의존하는 것을 막을 수 있습니다.

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 256), # 입력 784개를 256개 특징으로 변환
    nn.ReLU(),           # 비선형 활성화 함수 적용
    nn.Dropout(0.5),     # 학습 중 뉴런의 50%를 무작위로 비활성화
    nn.Linear(256, 10)   # 최종 10개 클래스 출력
)
```

드롭아웃의 핵심은 다음과 같습니다.

학습 시: 일부 뉴런을 무작위로 끄
평가 시: 모든 뉴런을 사용

드롭아웃 비율은 보통 0.2 ~ 0.5 범위에서 많이 사용합니다. 너무 높으면 학습이 잘 안 될 수 있습니다.

3.3 조기 종료(Early Stopping)

조기 종료는 검증 성능이 더 이상 좋아지지 않으면 학습을 멈추는 방법입니다.

오버피팅은 보통 학습을 너무 오래 할 때 발생합니다. 따라서 검증 손실이 일정 기간 동안 개선되지 않으면 학습을 중단합니다.

```
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',      # 검증 손실을 기준으로 확인
    patience=5,              # 5번의 epoch 동안 개선이 없으면 중단
    restore_best_weights=True # 가장 성능이 좋았던 시점의 가중치 복원
)
```

예를 들어 20번째 epoch에서 검증 손실이 가장 낮았고, 이후 5번 동안 좋아지지 않으면 학습을 멈추고 20번째 epoch의 모델 상태로 되돌립니다.

3.4 가중치 감쇠(Weight Decay, L2 Regularization)

가중치 감쇠는 모델의 가중치가 너무 커지지 않도록 제한하는 방법입니다.

가중치가 너무 크면 모델이 특정 입력에 과도하게 민감해질 수 있습니다. 이를 막기 위해 손실 함수에 가중치 크기에 대한 벌점을 추가합니다.

기본 손실 함수가 다음과 같다면,

Loss = 예측 오차

L2 정규화를 적용하면 다음과 같습니다.

Loss = 예측 오차 + $\lambda \times$ 가중치²의 합

여기서 λ 는 정규화 강도를 의미합니다.

```
import torch.optim as optim

optimizer = optim.AdamW(
    model.parameters(),
    lr=0.001,
    weight_decay=0.01 # 가중치가 너무 커지는 것을 방지
)
```

weight_decay=0.01은 모델이 학습 데이터에 지나치게 맞춰지지 않도록 가중치 크기를 억제하는 역할을 합니다.

3.5 모델 단순화

모델이 너무 복잡하면 오버피팅이 쉽게 발생합니다.

예를 들어 데이터가 1,000개뿐인데 수백만 개의 파라미터를 가진 모델을 사용하면 모델이 데이터를 외울 가능성이 큼니다.

모델 단순화 방법은 다음과 같습니다.

방법	설명
은닉층 수 줄이기	깊이를 줄임
뉴런 수 줄이기	각 층의 크기를 줄임
파라미터 수 줄이기	모델 용량 감소
더 작은 모델 사용	과도한 표현력 제한

예시입니다.

```
# 복잡한 모델
nn.Sequential(
    nn.Linear(784, 1024),
    nn.ReLU(),
    nn.Linear(1024, 512),
    nn.ReLU(),
    nn.Linear(512, 10)
)

# 단순한 모델
nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 10)
)
```

데이터가 적을수록 모델은 단순하게 설계하는 것이 안전합니다.

3.6 배치 정규화(Batch Normalization)

배치 정규화는 각 층의 입력 분포를 일정하게 맞춰주는 기법입니다.

학습 중 각 층으로 들어가는 값의 분포가 계속 바뀌면 학습이 불안정해질 수 있습니다. 배치 정규화는 평균과 분산을 조정해서 학습을 안정적으로 만듭니다.

```
model = nn.Sequential(
    nn.Linear(784, 256),
    nn.BatchNorm1d(256), # 256개 출력값을 정규화
    nn.ReLU(),
    nn.Linear(256, 10)
)
```

배치 정규화는 원래 학습 안정화 목적이 크지만, 약한 정규화 효과도 있어서 오버피팅 완화에 도움이 될 수 있습니다.

3.7 데이터 추가

가장 근본적인 해결 방법은 학습 데이터를 늘리는 것입니다.

오버피팅은 데이터가 부족할 때 자주 발생합니다. 데이터가 많아지면 모델이 특정 샘플을 외우기보다 일반적인 패턴을 학습할 가능성이 높아집니다.

데이터 추가 방법은 다음과 같습니다.

방법	설명
실제 데이터 추가 수집	가장 좋은 방법
공개 데이터셋 활용	비슷한 문제의 데이터 활용
데이터 증강	기존 데이터 변형
합성 데이터 생성	시뮬레이션 또는 생성 모델 활용

3.8 전이학습(Transfer Learning)

전이학습은 이미 대규모 데이터로 학습된 모델을 가져와서 내 문제에 맞게 다시 학습하는 방법입니다.

이미지 분류에서는 ResNet, EfficientNet, VGG 같은 사전학습 모델을 자주 사용합니다.

```
import torchvision.models as models
import torch.nn as nn

# ImageNet으로 미리 학습된 ResNet18 모델 불러오기
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)

# 기존 출력층을 현재 문제의 클래스 수에 맞게 교체
model.fc = nn.Linear(model.fc.in_features, 2)
```

데이터가 적을 때 처음부터 모델을 학습하면 오버피팅이 쉽게 발생합니다. 하지만 전이학습을 사용하면 이미 일반적인 특징을 알고 있는 모델에서 시작하므로 적은 데이터로도 좋은 성능을 얻기 쉽습니다.

3.9 교차 검증(Cross Validation)

교차 검증은 데이터를 여러 부분으로 나누어 여러 번 학습과 검증을 반복하는 방법입니다.

예를 들어 5-Fold 교차 검증은 데이터를 5개로 나누고, 매번 1개는 검증용, 나머지 4개는 학습용으로 사용합니다.

```
1회차: 1번 검증, 2~5번 학습
2회차: 2번 검증, 1,3,4,5번 학습
3회차: 3번 검증, 1,2,4,5번 학습
...
```

교차 검증은 데이터가 적을 때 모델의 성능을 더 안정적으로 평가하는 데 도움이 됩니다.

3.10 라벨 스무딩(Label Smoothing)

라벨 스무딩은 정답 라벨을 너무 확신하지 않도록 약간 부드럽게 만드는 방법입니다.

일반적인 분류 문제에서는 정답 클래스가 다음과 같이 표현됩니다.

```
정답: [0, 0, 1, 0]
```

라벨 스무딩을 적용하면 다음처럼 바뀝니다.

```
정답: [0.025, 0.025, 0.925, 0.025]
```

즉, 정답 클래스도 100% 확신하지 않고 약간의 여유를 줍니다. 이렇게 하면 모델이 학습 데이터의 정답을 너무 강하게 외우는 것을 줄일 수 있습니다.

```
loss_fn = nn.CrossEntropyLoss(label_smoothing=0.1)
```

4. 주요 기법 비교

기법	핵심 목적	적용 상황
데이터 증강	데이터 다양성 증가	이미지, 음성, 텍스트
드롭아웃	특정 뉴런 의존 방지	완전연결층, 일부 CNN
조기 종료	과도한 학습 방지	거의 모든 학습
Weight Decay	가중치 크기 억제	대부분의 딥러닝 모델
모델 단순화	모델 복잡도 감소	데이터가 적을 때
배치 정규화	학습 안정화	CNN, DNN
데이터 추가	일반화 성능 향상	가장 근본적 해결
전이학습	사전 지식 활용	데이터가 적을 때
교차 검증	평가 안정화	데이터가 적을 때
라벨 스무딩	과도한 확신 방지	분류 문제

5. 실무에서 자주 사용하는 조합

실무에서는 하나의 방법만 쓰기보다 여러 방법을 함께 사용합니다.

이미지 분류에서는 보통 다음 조합을 많이 사용합니다.

데이터 증강 + 전이학습 + Weight Decay + Early Stopping

일반적인 딥러닝 분류 모델에서는 다음 조합을 사용할 수 있습니다.

Dropout + BatchNorm + Weight Decay + Early Stopping

데이터가 적은 경우에는 다음 순서로 접근하는 것이 좋습니다.

1. 데이터 증강 적용
2. 전이학습 사용
3. 모델 크기 줄이기
4. Dropout 추가
5. Weight Decay 적용
6. Early Stopping 적용

6. 간단한 PyTorch 예시

```
import torch
import torch.nn as nn
import torch.optim as optim

# 오버피팅 방지를 적용한 간단한 신경망 모델 정의
class RegularizedModel(nn.Module):
    def __init__(self):
        super().__init__()

        # 입력층에서 은닉층으로 변환
        self.fc1 = nn.Linear(784, 256)

        # 배치 정규화: 은닉층 출력값의 분포를 안정화
        self.bn1 = nn.BatchNorm1d(256)

        # 드롭아웃: 학습 중 일부 뉴런을 무작위로 비활성화
        self.dropout = nn.Dropout(p=0.5)

        # 출력층: 10개 클래스로 분류
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        # 이미지를 1차원 벡터로 변환
        x = x.view(x.size(0), -1)

        # 첫 번째 선형 변환
        x = self.fc1(x)

        # 배치 정규화 적용
        x = self.bn1(x)

        # ReLU 활성화 함수 적용
        x = torch.relu(x)
```

```

        # 드롭아웃 적용
        x = self.dropout(x)

        # 출력층 계산
        x = self.fc2(x)

        return x

# 모델 생성
model = RegularizedModel()

# 라벨 스무딩이 적용된 손실 함수
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

# AdamW는 Weight Decay를 안정적으로 적용할 수 있는 최적화 함수
optimizer = optim.AdamW(
    model.parameters(),
    lr=0.001,
    weight_decay=0.01
)

```

이 코드에는 다음 오버피팅 방지 기법이 포함되어 있습니다.

```

BatchNorm
Dropout
Label Smoothing
Weight Decay

```

딥러닝 오버피팅 방지는 모델이 학습 데이터를 외우지 않고 새로운 데이터에도 잘 작동하도록 만드는 과정입니다.

가장 중요한 방향은 다음과 같습니다.

데이터를 다양하게 만든다.

모델이 너무 복잡하지 않게 한다.

학습을 너무 오래 하지 않는다.

가중치가 과도하게 커지지 않게 한다.

검증 성능을 기준으로 모델을 관리한다.

실무에서는 먼저 **Train Loss와 Validation Loss의 차이**를 확인하고, 차이가 커진다면 **데이터 증강, Dropout, Weight Decay, Early Stopping, 모델 단순화**를 순서대로 적용하는 것이 좋습니다.

2. 딥러닝 최적 모델 학습

딥러닝에서 **최적 모델 학습**이란 주어진 데이터에 대해 **일반화 성능(Generalization Performance)** 이 가장 높은 모델을 만드는 과정을 의미합니다.

즉,

학습 데이터에서는 높은 정확도
검증 데이터에서도 높은 정확도
새로운 데이터에서도 높은 정확도

를 동시에 만족하는 모델을 만드는 것이 목표입니다.

1. 최적 모델 학습의 의미

딥러닝 모델은 다음과 같은 과정을 통해 학습됩니다.

데이터 입력
↓
순전파(Forward Propagation)
↓
손실 계산(Loss)
↓
역전파(Backpropagation)
↓
가중치 갱신(Optimizer)
↓
반복 학습(Epoch)

이 과정에서 모델이 너무 단순하면 **언더피팅(Underfitting)** 이 발생하고,
너무 복잡하면 **오버피팅(Overfitting)** 이 발생합니다.

최적 모델은 그 중간 지점에 존재합니다.

2. 최적 모델의 조건

(1) 낮은 손실값

손실 함수(Loss Function)는 예측 오차를 의미합니다.

예를 들어

실제값 = 100
예측값 = 95

이면 오차가 존재합니다.

회귀 문제에서는 다음과 같은 MSE 를 사용합니다.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

여기서

- y : 실제값
- $\hat{y}$: 예측값

입니다.

손실값이 낮을수록 모델 성능이 좋습니다.

(2) 높은 정확도

분류 문제에서는 정확도를 사용합니다.

$$Accuracy = \frac{\text{정확히 예측한 개수}}{\text{전체 데이터 수}}$$

예시

100개 중 95개 정답

Accuracy = 95%

(3) 일반화 성능

가장 중요한 요소입니다.

예를 들어

```
Train Accuracy = 99%
```

```
Validation Accuracy = 98%
```

좋은 모델입니다.

반면

```
Train Accuracy = 99%
```

```
Validation Accuracy = 70%
```

오버피팅 상태입니다.

3. 최적 모델 학습 절차

단계 1. 데이터 수집

좋은 데이터가 좋은 모델을 만듭니다.

대표 데이터셋

분야	데이터셋
이미지	MNIST
이미지	CIFAR-10
이미지	ImageNet
텍스트	IMDB
시계열	Airline Passenger

단계 2. 데이터 탐색(EDA)

데이터의 특성을 확인합니다.

확인 내용

결측치
이상치
데이터 분포
클래스 불균형

예시

```
df.info()

df.describe()

df.isnull().sum()
```

단계 3. 데이터 전처리

정규화(Normalization)

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

결과

0~1 범위

표준화(Standardization)

$$z = \frac{x - \mu}{\sigma}$$

결과

평균 0
표준편차 1

단계 4. 데이터 분할

일반적으로

```
Train : 70%  
Validation : 15%  
Test : 15%
```

또는

```
Train : 80%  
Validation : 10%  
Test : 10%
```

을 사용합니다.

4. 모델 설계

예시

```
nn.Sequential(  
    nn.Linear(784,256),  
    nn.ReLU(),  
    nn.Linear(256,128),  
    nn.ReLU(),  
    nn.Linear(128,10)  
)
```

구성

```
입력층  
은닉층  
출력층
```

5. 손실 함수 선택

회귀 문제

MSE

$$MSE = \frac{1}{N} \sum (y - \hat{y})^2$$

MAE

$$MAE = \frac{1}{N} \sum |y - \hat{y}|$$

분류 문제

Binary Cross Entropy

$$BCE = -y \log(p) - (1 - y) \log(1 - p)$$

Cross Entropy

$$CE = - \sum y_i \log(\hat{y}_i)$$

6. 최적화 알고리즘 선택

SGD

$$w = w - \eta \nabla J(w)$$

특징

단순
느림

Momentum

$$v_t = \beta v_{t-1} - \eta \nabla J$$

특징

관성 적용

수렴 속도 향상

RMSProp

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)g_t^2$$

특징

학습률 자동 조절

Adam

가장 많이 사용됩니다.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$$

$$w_t = w_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

AdamW

Adam + Weight Decay

```
optimizer =  
torch.optim.AdamW(  
    model.parameters(),  
    lr=0.001,  
    weight_decay=0.01  
)
```

실무에서 매우 많이 사용됩니다.

7. 하이퍼파라미터 최적화

대표적인 하이퍼파라미터

항목	의미
Learning Rate	학습률
Batch Size	배치 크기
Epoch	반복 횟수
Dropout	드롭아웃
Weight Decay	정규화

Learning Rate

너무 작은 경우 : 학습 매우 느림

너무 큰 경우 : 발산

일반적 시작값 : 0.001

Batch Size

일반적 사용

32

64

128

256

8. 오버피팅 방지

Dropout

```
nn.Dropout(0.5) # 50% 뉴런 제거
```

BatchNorm

```
nn.BatchNorm1d(128) # 학습 안정화
```

Weight Decay

```
weight_decay=0.01 # 가중치 크기 제한
```

Data Augmentation

회전
확대
축소
좌우반전

Early Stopping

```
patience=5 # 검증 성능이 개선되지 않으면 중단
```

9. 모델 평가

대표 평가 지표

Accuracy

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

Precision

$$Precision = \frac{TP}{TP + FP}$$

Recall

$$Recall = \frac{TP}{TP + FN}$$

F1 Score

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$

10. 최적 모델 선택 기준

실무에서는 보통 다음 순서로 판단합니다.

① Validation Loss 최소

가장 중요

↓

② Validation Accuracy 최대

분류 문제

↓

③ Test 성능 확인

최종 일반화 성능

↓

④ 모델 저장

```
torch.save(  
    model.state_dict(),  
    "best_model.pth"  
)
```

11. 실무에서 가장 많이 사용하는 최적 학습 조합

이미지 분류

```
ResNet50
+
AdamW
+
CrossEntropyLoss
+
BatchNorm
+
Dropout
+
Data Augmentation
+
EarlyStopping
```

텍스트 분류

```
BERT
+
AdamW
+
CrossEntropyLoss
+
Weight Decay
+
EarlyStopping
```

시계열 예측

```
LSTM
+
Adam
+
MSELoss
+
EarlyStopping
```


정리

최적 모델 학습은 단순히 학습 정확도를 높이는 것이 아니라,

1. 좋은 데이터 확보
2. 적절한 전처리
3. 적절한 모델 설계
4. 올바른 손실함수 선택
5. 적절한 최적화 알고리즘 사용
6. 하이퍼파라미터 튜닝
7. 오버피팅 방지
8. 검증 데이터 기반 모델 선택

을 통해 새로운 데이터에서도 높은 성능을 유지하는 모델을 만드는 과정입니다.

실무에서는 다음 조합이 가장 많이 사용됩니다.

```
Optimizer : AdamW  
Loss : CrossEntropyLoss  
Regularization : Weight Decay + Dropout  
Normalization : BatchNorm  
Overfitting Prevention : EarlyStopping  
Model Selection : Lowest Validation Loss
```

이 조합이 현재 딥러닝 분류 문제에서 가장 널리 사용되는 최적 학습 전략 중 하나입니다.